PRODUCT REQUIREMENTS DOCUMENT

Agent-Based Version Control System

Final Year Engineering Project

Project Type	Final Year Engineering Project
Domain	Artificial Intelligence, Software Engineering, Version Control, Multi-Agent Systems
Document Purpose	Master product and development requirements
Version	1.0
Date	August 2026

AI understands. AI recommends. Humans decide. Git executes.

1. Project Overview

The **Agent-Based Version Control System (ABVCS)** is an intelligent software development platform that combines traditional Git-based version control with specialised AI agents. The system assists developers in managing repositories, understanding changes, generating meaningful commits, reviewing code, running tests, analysing merge conflicts, and explaining project history.

The system will not attempt to replace Git. Instead, it will provide an intelligent agent layer around a Git-based version-control engine. A **Human-in-the-Loop** design will ensure that AI can analyse and recommend actions while the developer remains in control of critical repository operations.

2. Problem Statement

- Developers must manually inspect large code changes and diffs.
- Writing useful and consistent commit messages is repetitive.
- Code review requires time and can miss issues.
- Choosing and running appropriate tests after changes is often manual.
- Complex merge conflicts can be difficult to understand and resolve.
- Repository history can be difficult to interpret in large projects.
- Traditional version-control tools do not natively provide a coordinated multi-agent assistance layer.

3. Proposed Solution

The proposed platform will combine Git-based version control with an AI agent layer. The system will provide repository management, branch and merge operations, diff analysis, AI-generated commits, code review, test execution, conflict-resolution assistance, natural-language history analysis, and agent activity logging.

4. Project Objectives

- Develop a functional Git-based version-control platform.
- Provide a web interface for repository management.

- Develop specialised AI agents for version-control tasks.
- Automatically analyse source-code changes.
- Generate meaningful commit messages using AI.
- Provide AI-assisted code reviews.
- Execute and report project tests.
- Assist developers in understanding and resolving merge conflicts.
- Provide natural-language explanations of repository history.
- Implement human approval before critical Git operations.
- Maintain logs of AI-agent activities and decisions.
- Keep the architecture modular so individual agents can be developed and tested independently.

5. Scope

5.1 In Scope

- **Version control:** repository creation, status, staging, commits, history, diffs, branches, merging and conflict detection.
- **AI assistance:** change analysis, commit-message generation, code review, test-result analysis, conflict explanation, resolution suggestions and history explanation.
- **Agent system:** orchestrator, specialised agents, controlled tools, result validation, human approval and agent logs.
- **User interface:** dashboard, repository explorer, diff viewer, branch management, AI assistant, review and conflict interfaces.

5.2 Out of Scope

- Reimplementing the complete Git system from scratch.
- Training a proprietary large language model.
- Guaranteeing automatic resolution of every possible merge conflict.
- Executing arbitrary repository code directly on the host machine.
- Supporting every programming language and build system.
- Building a production-scale GitHub/GitLab replacement.
- Giving AI unrestricted autonomous control over repositories.

6. Target Users

- **Software developers:** users who manage repositories, branches, commits, reviews and merges.
- **Student development teams:** teams collaborating on academic software projects.
- **Project supervisors:** users who may inspect commits, reviews, tests and agent activity.

7. Core Product Workflow

```
Create / open repository
|
v
Modify source code
|
v
View status + diff
```

```
|
v
AI analyses changes
|
+--> Generate commit message
|
+--> Review code
|
+--> Run tests
|
v
Human reviews recommendations
|
v
Commit / branch / merge
|
v
If conflict occurs:
Conflict Agent suggests resolution
|
v
Human approves or edits
|
v
Repository updated
```

8. Major System Modules

8.1 Repository Management

Create, open and manage repositories; display repository metadata and status.

8.2 Git Engine

Provide init, status, add, commit, log, diff, branch, checkout and merge operations using Git rather than recreating Git.

8.3 Repository Explorer

Browse directories and source files through the web interface.

8.4 Change Analysis Agent

Analyse diffs, changed files, repository context, impact and risk.

8.5 Commit Agent

Generate meaningful commit messages and allow accept, edit or reject before committing.

8.6 Code Review Agent

Identify possible bugs, code smells, security issues, maintainability problems and edge cases.

8.7 Test Agent

Detect supported test frameworks, execute tests in isolation and explain results.

8.8 Conflict Detection

Detect merge conflicts and display conflicting sections clearly.

8.9 Conflict Resolution Agent

Analyse conflicts and suggest possible resolutions while requiring human approval.

8.10 Repository History Agent

Answer natural-language questions using commits, diffs and repository history.

8.11 Agent Orchestrator

Route tasks to specialised agents and prevent unnecessary AI calls.

9. Human-in-the-Loop Design

Human approval is a fundamental safety and reliability requirement. AI-generated actions will not automatically perform critical or destructive repository operations.

- Commit suggestions require user approval.
- Merge and conflict-resolution suggestions require user approval.
- AI recommendations must be distinguishable from confirmed system results.
- Users can reject, edit or manually perform critical operations.

```
AI Recommendation
|
v
Response Validation
|
v
Human Approval
|
v
Git Operation
|
v
Audit Log
```

10. AI Agent Tool Architecture

Agents will interact with repositories through explicitly defined tools. The LLM will not receive unrestricted operating-system access.

```
get_repository_status()
get_git_diff()
get_file_content()
get_commit_history()
compare_branches()
run_tests()
create_branch()
create_commit()
get_conflict()
```

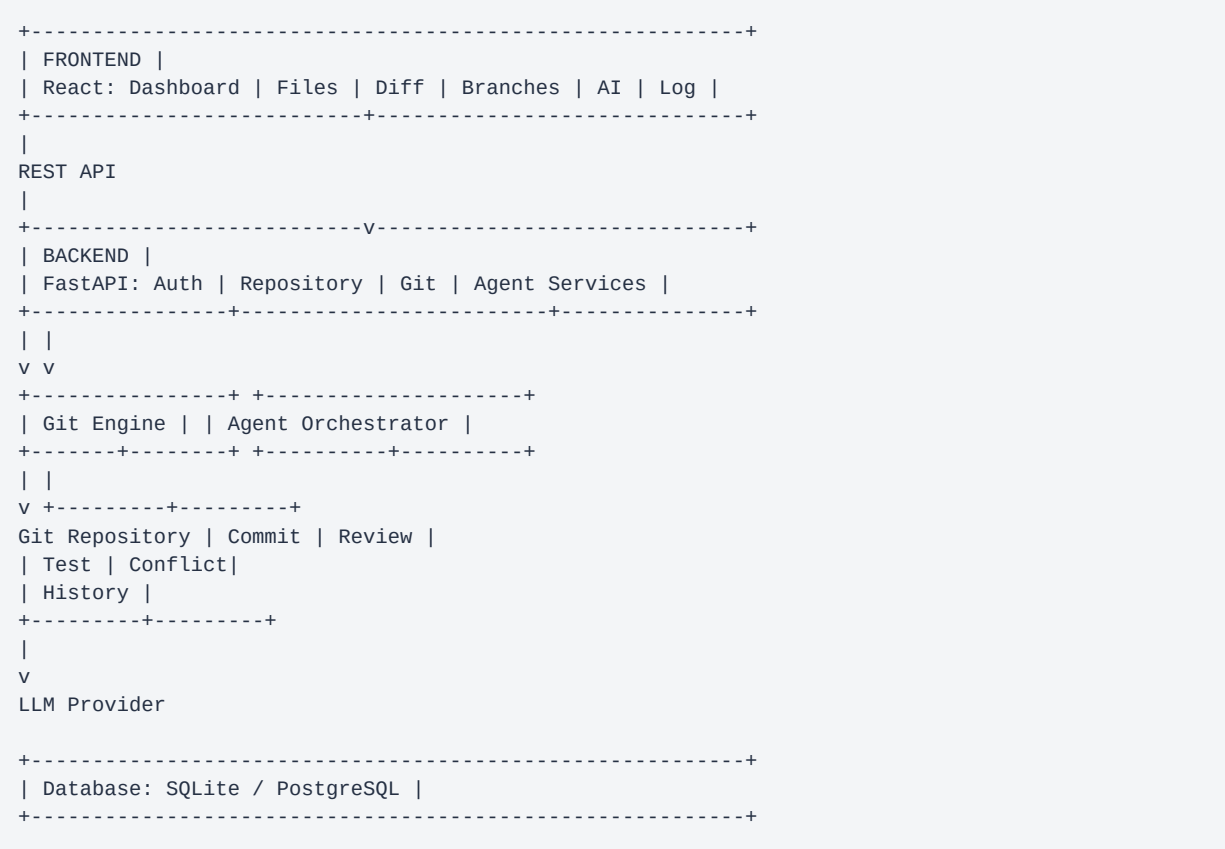
Design principle: AI reasoning -> approved tools -> repository operation.

11. Technology Stack

Layer	Recommended Technology	Purpose
Frontend	React	Dashboard, repository explorer, diff viewer and AI interfaces
Backend	Python + FastAPI	REST API and application logic
Version Control	Git	Underlying repository and version history

Layer	Recommended Technology	Purpose
Git Integration	GitPython / Dulwich / Git CLI	Controlled Git operations from Python
AI	LLM API	Reasoning, summarisation, review and agent capabilities
Database	SQLite initially	Application metadata, agent runs and audit records
Extended DB	PostgreSQL (optional)	Scalable deployment if required
Sandbox	Docker	Isolated test/code execution
API	REST	Frontend-backend communication

12. High-Level Architecture



13. Security Requirements

- Repository code must not be executed directly on the host system.
- Test execution should use a sandbox such as Docker.
- AI tools must be explicitly defined and permission controlled.
- AI-generated structured responses must be validated before use.
- Critical operations must require human approval.
- API keys and credentials must be stored securely and never hard-coded.
- Uploaded repository content must be validated.

- Path traversal and unauthorised file access must be prevented.
- Repository and agent operations should be logged for auditing.

14. Functional Requirements

ID	Requirement
FR-01	Users shall be able to authenticate securely.
FR-02	Users shall be able to create and manage repositories.
FR-03	Users shall be able to browse repository files.
FR-04	The system shall display modified, added, deleted and untracked files.
FR-05	The system shall display code differences.
FR-06	The system shall generate AI-assisted commit messages.
FR-07	Users shall approve, edit or reject AI-generated commits.
FR-08	Users shall create, switch, compare and merge branches.
FR-09	The system shall provide AI-assisted code review.
FR-10	The system shall execute supported project tests in isolation.
FR-11	The system shall explain test results in human-readable language.
FR-12	The system shall detect merge conflicts.
FR-13	The system shall generate AI-assisted conflict-resolution suggestions.
FR-14	Users shall be able to ask natural-language questions about repository history.
FR-15	The system shall maintain agent activity and operation logs.

15. Non-Functional Requirements

- **Performance:** normal Git operations should return with minimal delay; AI operations may take longer.
- **Reliability:** AI-service failure must not disable core Git functionality.
- **Modularity:** each agent should be independently testable and replaceable.
- **Maintainability:** frontend, backend, Git, AI and database layers should remain separated.
- **Security:** repository access and code execution must be controlled.
- **Scalability:** additional agents should be addable without major architectural changes.
- **Usability:** the interface should be understandable to users familiar with basic Git.

16. MVP Definition

If development time becomes constrained, the following minimum product must remain functional:

- Repository creation and management
- Status, diff, commit and history

- Basic branch management
- Web dashboard and repository explorer
- Change Analysis Agent
- Commit Agent
- Code Review Agent

17. Target Final Version

- Git-based repository management
- Web dashboard
- Branch management
- Diff viewer
- Change Analysis Agent
- Commit Agent
- Code Review Agent
- Test Agent
- Test-result analysis
- Conflict detection
- Conflict Resolution Agent
- Repository History Agent
- Agent Orchestrator
- Human approval workflow
- Agent activity logs
- Authentication
- Sandboxed test execution

18. Development Roadmap

Phase	Main Work	Working Checkpoint
1	Git foundation: repository, status, diff, commit, log, branch, merge	Working Git backend
2	FastAPI backend, APIs, database and authentication	Working REST backend
3	React dashboard, files, diff, history and branches	Working web application
4	Commit Agent	First working AI agent
5	Code Review Agent	AI review report
6	Test Agent + sandbox	Automated test report
7	Conflict Agent	AI conflict suggestion

Phase	Main Work	Working Checkpoint
8	Agent Orchestrator	Coordinated multi-agent system
9	Security, validation, fallbacks and audit logs	Reliable integrated system
10	Testing, evaluation and documentation	Final project release

19. Failure and Fallback Strategy

A core architecture principle is: **AI failure must not equal system failure.**

Component	Primary	Fallback
Commit	AI-generated message	Manual commit message
Code Review	AI review	Static analysis / manual review
Testing	Agent-assisted test selection and analysis	Manual test command
Conflict	AI resolution suggestion	Normal Git conflict editor
AI API	LLM service	Core VCS remains available

20. Major Risks and Mitigation

Risk	Impact	Mitigation
Git complexity	High	Use existing Git implementation and isolate Git integration.
AI hallucination	High	Schema validation, confidence indicators and human approval.
AI/API failure	Medium	Fallback to normal Git/manual operations.
Unsafe code execution	High	Sandbox test execution using Docker.
LLM cost	Medium	Invoke AI only where it adds value; avoid AI for basic Git operations.
Complex merge conflicts	High	Provide assisted suggestions, not guaranteed autonomous resolution.
Frontend/backend integration	Medium	Develop and test APIs independently before UI integration.
Scope creep	High	Freeze MVP and treat advanced features as stretch goals.
Time constraints	High	Maintain working checkpoints after every phase.

21. Evaluation Metrics

- **Commit Agent:** relevance and quality of generated messages; user acceptance rate.
- **Code Review Agent:** useful issues identified, false-positive rate and developer evaluation.
- **Test Agent:** successful execution and correctness of result interpretation.

- **Conflict Agent:** usefulness and correctness of suggested resolutions; human acceptance rate.
- **Overall system:** task completion time, usability, reliability, response time and failure recovery.

22. End-to-End Success Scenario

```
1. Create repository
2. Add source code
3. Modify files
4. View status and diff
5. Ask AI to analyse changes
6. Generate commit message
7. Review AI suggestion
8. Approve commit
9. Create feature branch
10. Modify branch
11. Run tests
12. Merge branch
13. Detect merge conflict
14. Receive AI conflict suggestion
15. Approve or edit resolution
16. Complete merge
17. Ask AI about repository history
```

23. Key Design Principles

1. Git remains the source of truth. AI assists Git; it does not replace it.

2. AI assists, humans decide. Critical operations require explicit approval.

3. Agents use controlled tools. No unrestricted operating-system access.

4. Modular architecture. Agents can be added, removed or replaced independently.

5. Graceful degradation. The application remains useful when AI services fail.

6. Build incrementally. Every phase must produce a working checkpoint.

7. Security before autonomy. Repository and code-execution permissions are deliberately restricted.

24. Final Project Vision

The final product will be a web-based intelligent version-control platform that helps developers understand and manage source-code changes while preserving the reliability and control of Git.

The project demonstrates the practical integration of Artificial Intelligence, Multi-Agent Systems, Software Engineering, Version Control, Web Development, APIs, Databases, Security and DevOps concepts into a single working system.

AI understands. AI recommends. Humans decide. Git executes.